

Evaluating ML-KEM at (much) higher security levels

Anonymous Submission

Abstract.

Keywords: No keywords given.

1 Introduction

After a multi-year, multi-round public competition, NIST published its first post-quantum standards FIPS-203 [29], FIPS-204 [28], and FIPS-205 [30] in 2024, specifying ML-KEM, ML-DSA, and SLH-DSA, respectively. Later in 2025, NIST selected an additional KEM, HQC, for standardization in the fourth round [2]. Unlike ML-KEM, which is based on module learning with errors (LWE), HQC relies on the Quasi-Cyclic Syndrome Decoding (QCSD) problem [21], providing diversity beyond lattice-based assumptions. This choice also marked a shift away from the lattice-based KEMs that dominated the Round 3 submissions. Four of the schemes in the PQC Round 3 Submissions [27] – KYBER [12] (standardized as ML-KEM), Saber [15], NTRU LPRime [10], and FrodoKEM [11] – are closely related lattice KEMs, spanning a spectrum of algebraic structure, from FrodoKEM’s unstructured LWE, through KYBER’s and Saber’s cyclotomic modules, to the prime-degree number fields of NTRU Prime. This spectrum reflects different design principles, commonly understood as structure vs. conservatism. ML-KEM’s module structure gives small keys and fast NTT arithmetic, but is also seen as a potential attack surface, so conservative designs discard it entirely [11]. FrodoKEM, for example, is based on a more conservative hardness assumption of plain LWE, one of the reasons why it was recommended by bodies such as BSI, ENISA, and NLNCSA [33] and recently standardized by ISO [23].

It is useful to separate the two notions of conservatism this framing bundles together as they concern two different classes of attack. The first arises from the choice of hardness assumption. FrodoKEM rests on unstructured LWE to avoid a hypothetical structure break, an attack that would exploit the algebraic structure of ring/module LWE but not apply directly to plain LWE. The second involves the security margin a scheme carries above its target level, which protects against advances in the best known attacks today. This could include improved lattice reduction, dual attacks, or sharper cost models that would affect the concrete security of every lattice scheme at its current parameters, including FrodoKEM. Both of these hedges are distinct, but not entirely independent because additional parameter margin can also increase the cost of a hypothetical structural attack, and removing algebraic structure can affect the cost of known attacks. A third, orthogonal hedge is to change the hardness assumption altogether, as HQC does by relying on QCSD instead of LWE. This provides diversity, and at first glance, some additional assurance since a break of LWE could still leave HQC secure. However, this cryptanalytic independence is not absolute. Recent work has demonstrated connections between lattice and code-based cryptanalysis, including adaptations of lattice-sieving techniques to codes [17]. The quasi-cyclic structure of HQC has also been noted to have important parallels with the ring and module structures used in structured lattice schemes [31]. These approaches address different (hypothetical) ways things could go wrong: unstructured LWE protects

against attacks that exploit algebraic structure, additional parameter margin protects against improvements to existing attacks, and QCSD provides diversity against a failure of the LWE assumption itself. At this time, it is impossible to predict which one of these breakthroughs is more likely to occur. Faced with this uncertainty, increasing the margin has been a historically standard response, provided that sufficient bandwidth is available.

For computationally secure schemes, a concrete security estimate is the cost of the best attack known at the time, and such estimates evolve as cryptanalysis matures. For instance, the recommended size of an RSA modulus climbed from 512 to 1024 to 2048 bits and beyond as factoring algorithms and available computation advanced [9]. Lattice estimates are not static either. The cost models used for estimation and the attacks have evolved over more than a decade [14, 3]. The core framework for estimating the security of LWE-based schemes has nevertheless remained stable, with BKZ-based lattice reduction at its core. Within this framework, the cost of the best known attacks continues to drift in both directions as improvements are proposed [20, 25], scrutinized [18], and refined [13].

If the best known attacks continue to improve, how much margin is required remains open, and the useful question is not whether today’s KEMs carry enough but how much margin they can carry at all. This reframing matters because additional margin comes at a cost, requiring larger parameters and hence higher bandwidth and runtime, so no scheme can indefinitely withstand advances in cryptanalysis simply by scaling its parameters. For LWE-family KEMs there is a further constraint, since the parameters that determine security also control the decryption failure probability, so increasing the noise to gain a larger security margin necessarily affects correctness.

Crucially, what structure determines is the *price* of buying additional margin. At matched security levels, an unstructured scheme already requires roughly an order of magnitude more bandwidth than a module scheme [11, 12]. Since these communication costs are considered practical enough for conservative deployments, a module scheme can spend the same bandwidth budget on larger parameters and obtain a conservative security margin while retaining the efficiency advantages of NTT arithmetic.

Recent work has explored this high-margin regime from the perspective of unstructured LWE. KyFrog [26] increases the LWE dimension and adjusts the noise distribution to achieve substantially higher security estimates for much larger ciphertexts. This leaves open the corresponding question for module-based KEMs:

How much additional security margin can their algebraic structure buy within the communication budget of an unstructured LWE scheme, and at what computational cost?

We answer this question with KAIBURR, a module KEM that scales ML-KEM’s module rank and introduces a generalized noise distribution to preserve correctness. We first characterize the correctness and security limits of naively increasing ML-KEM’s module rank, then introduce a noise family that allows further scaling. We then build three different parameter sets with the new noise sampler, and implement them across conventional software and dedicated hardware-accelerated platforms. Finally, we compare KAIBURR with existing KEMs, including FrodoKEM and HQC. More specifically, our contributions can be summarized as follows.

1.1 Our contributions

- *We show that naively scaling ML-KEM’s module rank hits a correctness wall.* Raising k alone, with KYBER’s noise and modulus fixed, satisfies the 2^{-128} decryption-failure bound only up to $k = 7$ and fails already at $k = 8$, so larger module rank cannot buy additional security margin on its own (Section 3).
- *We introduce a one-parameter noise family that allows us to scale further.* Our family $f(t)$ generalizes KYBER’s centered binomial distribution, thinning its ± 2 tail geometrically as t grows while leaving the ± 1 mass unchanged. The parameter t can

- 93 be tuned against module rank k to keep decryption failure below 2^{-128} as k increases
 94 (Section 4).
- 95 • *We select concrete parameter sets that reach large security margins within FrodoKEM-*
 96 *640's communication budget.* Using this trade-off, we obtain three parameter sets,
 97 kaiburr-1792, -4608, and -6144, at $k = 7, 18, 24$, whose classical/quantum Core-SVP
 98 security climbs from 479/434 to 1717/1557 bits while their public keys and ciphertexts
 99 stay within the communication budget of FrodoKEM-640, its smallest standardized
 100 parameter set (Section 7).
 - 101 • *We build KAIBURR across three implementation targets.* We implement all three
 102 parameter sets in C, in Jasmin with machine-checked constant-time verification,
 103 and as an ACC variant targeting the Pavona hardware root of trust, detailing
 104 the engineering consequences of scaling k and removing ciphertext compression
 105 (Section 5).
 - 106 • *We give a formally verified bound on Kaiburr's decryption-failure probability.* Building
 107 on the EasyCrypt framework of [8], we show that removing ciphertext compression
 108 is what makes this bound provable rather than heuristic and close a gap that exists
 109 for ML-KEM's published bound by removing compression (Section 6).
 - 110 • *We benchmark KAIBURR against FrodoKEM and HQC.* Across three Intel microar-
 111 chitectures, we show that our AVX2 and Jasmin implementations run substantially
 112 faster than FrodoKEM-640 and achieve performance comparable to HQC, despite
 113 having (much) larger security margins. (Section 7).

114 1.2 Access to artifacts

115 All software, including parameter analysis scripts, implementation source code, and other
 116 artifacts, is publicly available on [GitHub](#).

117 2 Preliminaries

118 **Terminology** Throughout the paper, we use KYBER and ML-KEM interchangeably,
 119 except when distinguishing the original KYBER submission from the standardized ML-KEM
 120 specification.

121 **Ring and module setup.** Let $n = 256$ and $q = 3329$, and let $R = \mathbb{Z}[X]/(X^n + 1)$ and
 122 $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ be KYBER's base ring, with coefficients centered in $(-q/2, q/2]$. The
 123 modulus satisfies $n \mid (q - 1)$, needed for the number-theoretic transform (NTT) that KYBER
 124 uses for polynomial multiplication in R_q . A module rank k determines the dimensions
 125 of the public matrix $\mathbf{A} \in R_q^{k \times k}$ and the secret/error vectors $\mathbf{s}, \mathbf{e} \in R_q^k$. $k = 2, 3, 4$ give
 126 ML-KEM-512, -768, and -1024, respectively [29].

127 **Centered binomial noise.** KYBER samples secret and error coefficients from the centered
 128 binomial distribution CBD_η . It draws 2η uniform bits $a_1, \dots, a_\eta, b_1, \dots, b_\eta$ and outputs
 129 $\sum_{i=1}^\eta a_i - \sum_{i=1}^\eta b_i$, a value in $\{-\eta, \dots, \eta\}$ with variance $\eta/2$. ML-KEM-768 and -1024
 130 use CBD_2 for both the secret and error terms; ML-KEM-512 uses CBD_3 for the secret.
 131 We write $\text{CBD}_\eta(x)$ for the probability mass assigned to x by this distribution. CBD_η is
 132 especially desirable because it can be sampled from raw PRF output in constant time
 133 without rejection and uses only bit-counting and a subtraction.

Module-LWE. The scheme’s IND-CPA security reduces to the decisional Module Learning With Errors (MLWE) problem [24] over R_q . An adversary must distinguish $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$, where $\mathbf{A} \leftarrow R_q^{k \times k}$ and $\mathbf{s}, \mathbf{e} \leftarrow \text{CBD}_\eta^k$, from a uniformly random pair in $R_q^{k \times k} \times R_q^k$. Increasing k increases the dimension of the lattice problem and is the main lever KYBER uses to raise its claimed security level [5].

Core-SVP hardness estimates. We estimate MLWE hardness following the standard core-SVP methodology used throughout the KYBER submission documents and re-implemented in the KYBER team’s public estimator scripts. An adversary is modeled as running BKZ lattice reduction with block size b against the primal (or dual) lattice attack. The cost of BKZ- b is dominated by a single call to an SVP oracle in dimension b , whose cost is estimated as 2^{cb} operations for a constant c (e.g., $c \approx 0.292$ classically and $c \approx 0.265$ under a quantum sieve) [14, 3]. For a given parameter set, the scripts search over block size b and sample count m for the cheapest attack that succeeds against the secret/error distribution’s standard deviation, and report the $\log_2$ cost as the *classical* and *quantum* core-SVP security level. We report both throughout. We note that these estimates are conservative lower bounds on attack cost rather than concrete running times.

Decryption failure. Because KYBER’s public-key encryption scheme is only approximately correct, decapsulation can fail. The accumulated LWE and rounding error can push a coefficient of the decrypted message past the threshold at $q/4$ that separates a 0 bit from a 1 bit. We refer to the decryption-failure probability as the probability, over the randomness of key generation and encryption, that at least one of the n message-bit coefficients suffers such an error.¹ It is computed by convolving the noise, secret, and (when present) compression rounding-error distributions coordinate-wise and taking the tail probability of the final distribution beyond $q/4$, scaled by n . A failure probability of 2^{-128} is considered a standard benchmark which aligns with the target of 128-bit security. We use the same bound throughout as our correctness target.

From PKE to KEM. KYBER’s IND-CPA-secure public-key encryption scheme is lifted to an IND-CCA2-secure KEM using a tweaked variant of the Fujisaki–Okamoto (FO) transform [29]. The ciphertext produced by encryption consists of two components, $\mathbf{u} \in R_q^k$ and $v \in R_q$, which ML-KEM serializes after *compression*, where each coefficient is rounded. This trades a small increase in decryption-failure probability for a smaller ciphertext. The public key, consisting of $\mathbf{A}$ ’s seed and $\mathbf{A}\mathbf{s} + \mathbf{e}$, is never compressed. Throughout the paper, “ciphertext compression” refers to this rounding of $\mathbf{u}$ and v . We remove this compression in KAIBURR to achieve formally verified correctness bounds (see Section 6). The FO transform itself, the ring, the NTT, and the symmetric primitives are left untouched.

3 Warmup: Just changing k

The main mechanism to increase bit security in KYBER is to raise the module rank k [5]. Fixing the ML-KEM base ring, noise distribution, and modulus, and varying only k , we measure the decryption-failure probability using the scripts provided by KYBER team.² Throughout we use the centered binomial noise CBD_2 (the noise of ML-KEM-768 and -1024; recall that ML-KEM-512 uses CBD_3). The ranks $k = 3$ and 4 recover the uncompressed analogue of standardized sets ML-KEM-768, and -1024, while $k \geq 5$ is an extrapolation. Table 1 reports the results. Against the 2^{-128} failure bound expected at

¹Even though a single decryption error may not sound catastrophic, it is well established that once an attacker finds a single failure, they can use it as a compass to drastically accelerate finding more failures, a type of attack known as “directional failure-boosting” [16].

²<https://github.com/pq-crystals/security-estimates>

this security level, the bound holds at $k = 7$ ($2^{-133.7}$) but fails at $k = 8$ ($2^{-116.8}$), so raising k alone cannot buy higher security without sacrificing correctness.

These values are computed without ciphertext compression. This matches KAIBURR, which removes compression (Section 5.3), so the numbers are exact for our setting. If the compression from KYBER is retained, the rounding applied to $\mathbf{u}$ and v introduces additional error, which would increase the true failure probability, and the correctness limit on k would be reached even earlier.

Table 1: Decryption-failure probability and public-key and ciphertext sizes as a function of module rank k , at fixed ML-KEM noise and modulus, without ciphertext compression. Ranks $k = 3, 4$ correspond to uncompressed ML-KEM-768, and -1024.

k	Failure probability	pk (bytes)	ct (bytes)
3	$2^{-299.6}$	1184	1536
4	$2^{-231.1}$	1568	1920
5	$2^{-186.5}$	1952	2304
6	$2^{-155.9}$	2336	2688
7	$2^{-133.7}$	2720	3072
8	$2^{-116.8}$	3104	3456

4 Revisiting the noise distribution

Even though the resulting public key and ciphertext sizes at $k = 7$ remain relatively small, increasing k further is not possible without affecting correctness. Instead, one could reduce the failure probability at large k by increasing the modulus q . We note that all variants of KYBER fix $q = 3329$, which is explicitly justified in the submission documentation to NIST. The modulus $q = 3329$ is a suitable small prime satisfying the NTT-enabling condition $n \mid (q - 1)$, notably weaker than the $2n \mid (q - 1)$ one might expect, which KYBER deliberately does not require [6]. It was reduced from the round-1 value 7681 to recover the bandwidth spent by dropping public-key compression and co-tuned to balance ciphertext size, failure probability, and security.

Since ML-KEM’s standardization, highly optimized and high-assurance implementations have emerged across software and hardware platforms [32, 7, 1]. Changing q would mean not being able to reuse any existing optimized routines in software or hardware, which sacrifices an important engineering advantage and results in a different scheme altogether. We choose not to change q for this reason.

This is not to say that k is the only parameter that can be modified. We focus on changes that preserve the *essence* of KYBER: the ring $\mathbb{Z}_q[X]/(X^{256} + 1)$, the NTT implementation and its constants, and the existing analysis framework. The module rank k qualifies trivially. So does the error distribution. Since the noise only affects the secret and error terms and does not interact with the arithmetic structure or the NTT, it can be replaced without altering the core implementation. What it changes are only the MLWE hardness estimate and the decryption-failure probability, both of which can be recomputed through the same scripts, over the same ring.

In this sense, the error distribution is another part of the scheme alongside k that moves both security and correctness without changing what KYBER is at its core. This is the direction we pursue and where our main contribution lies.

To scale past $k = 7$, we need a noise knob that pushes the failure-driving part of the error back down as k grows, while preserving KYBER’s *essence*. Equally important, it should retain the cheap, branchless, constant-time sampling that motivates KYBER’s use

213 of a centered binomial in the first place. This requirement rules out simply using a discrete
 214 Gaussian, as in FrodoKEM.

215 4.1 A parameterized noise distribution

216 We introduce a one-parameter family $f(t)$ on $\{-2, -1, 0, 1, 2\}$, the same support as CBD_2 ,
 217 defined by

$$218 \quad \Pr[\pm 2] = 2^{-t}, \quad \Pr[\pm 1] = \frac{1}{4}, \quad \Pr[0] = \frac{1}{2} - 2^{-(t-1)}.$$

219 The mass at ± 1 is fixed. The single parameter t acts only on the ± 2 tail, thinning it
 220 geometrically. A sample is drawn from uniform bits $a_1, \dots, a_t$ as

$$221 \quad x = (2a_t - 1) \left(1 - a_1 + 2 \prod_{i=1}^{t-1} a_i \right),$$

222 which is branchless and therefore samples in constant time; we machine-check both constant-
 223 time (CT) and speculative constant-time (SCT) in Jasmin [4] (see Section 5). The standard
 224 deviation $\sigma(t) = \sqrt{0.5 + 2^{3-t}}$ decreases from 1 at $t = 4$ toward $\sqrt{0.5}$ as t grows.

Crucially, $f(t)$ generalizes KYBER's own noise distribution. At $t = 4$, the probabilities become

$$\Pr[\pm 2] = \frac{1}{16}, \quad \Pr[\pm 1] = \frac{1}{4}, \quad \Pr[0] = \frac{3}{8},$$

225 exactly matching CBD_2 , the noise distribution used by ML-KEM-768 and -1024. As
 226 $t \rightarrow \infty$, the ± 2 mass vanishes and $f(t)$ approaches CBD_1 .

227 4.2 Finer control over the noise

A key property of $f(t)$ is that increasing t reduces the probability of the ± 2 terms exponentially while leaving the ± 1 mass unchanged. As defined above, we have

$$\Pr_{f(t)}[\pm 2] = 2^{-t}, \quad \Pr_{f(t)}[\pm 1] = \frac{1}{4}.$$

228 So, t is the tunable parameter for thinning the tail of the distribution without changing
 229 the bulk ± 1 terms. This gives finer control over the noise than the standard centered
 230 binomial choices. In particular, CBD_1 and CBD_2 have standard deviations $\sqrt{0.5}$ and 1,
 231 respectively, with no centered binomial parameter providing an intermediate distribution.
 232 As shown above, $f(t)$ ranges from CBD_2 toward CBD_1 as t increases.

233 This finer control is useful as the module rank k increases. Increasing k increases the
 234 amount of error accumulated during encryption and therefore makes the decryption-failure
 235 constraint more restrictive. Rather than moving directly from CBD_2 to CBD_1 , we can
 236 increase t as k grows, reducing the ± 2 contribution while keeping the ± 1 contribution
 237 fixed.

238 4.3 Selecting t

239 Since t is the number of uniform bits per sample, vectorization-friendly values are natural,
 240 especially for AVX2 implementations (see Section 5). As shown above, $t = 4$ recovers
 241 the CBD_2 distribution, so it serves as our anchor. Notice that $t = 8$ consumes exactly
 242 one PRF byte per coefficient, meaning it admits one-byte-per-coefficient SIMD packing
 243 without cross-byte bit-splitting. This makes $t = 8$ a very convenient choice. Our third
 244 choice, $t = 6$, sits in the middle and does not share the byte-alignment of $t = 4$ or $t = 8$.
 245 Sampling four coefficients consumes $6 \times 4 = 24$ bits, i.e. three bytes of PRF output, which
 246 is not as natural to implement as the byte-aligned $t = 4$ and $t = 8$ cases. It is nonetheless
 247 interesting relative to $t = 8$, for two reasons. First, it retains more of the ± 2 tail, giving

a higher standard deviation and more security per dimension. Second, it spends fewer PRF bytes per coefficient (0.75 vs. 1), which is relevant on platforms *without* Keccak acceleration. For each choice of t , we pick the highest k satisfying the failure probability bound of 2^{-128} , staying within the communication budget of FrodoKEM640.

We use FrodoKEM-640 as our baseline for parameter selection and comparison throughout the paper because it is the smallest standardized FrodoKEM parameter set and therefore imposes the tightest communication budget of 9,616 public-key bytes and 9,720 ciphertext bytes. This gives us a restrictive baseline for asking how much security margin KAIBURR can achieve within the communication cost of a conservative lattice-based KEM.

The three parameter sets we carry forward are $f(4)$, $f(6)$, and $f(8)$, at $k = 7$, 18, and 24 respectively, which follow from these choices (Table 2).

Table 2: KAIBURR parameter sets. Classical/quantum security (Core-SVP) and failure probability computed with the KYBER team’s scripts; other parameters as in ML-KEM, compression removed.

Scheme	k	noise	failure	class.	quant.	pk (B)	ct (B)
kaiburr-1792	7	$f(4)$	$2^{-133.7}$	479	434	2720	3072
kaiburr-4608	18	$f(6)$	$2^{-134.1}$	1281	1162	6944	7296
kaiburr-6144	24	$f(8)$	$2^{-139.9}$	1717	1557	9248	9600

4.4 Sizes in context

Table 3 places the three KAIBURR parameter sets alongside all standardized FrodoKEM and HQC parameter sets. All three KAIBURR variants, including kaiburr-6144, remain within the communication budget of FrodoKEM-640, the smallest FrodoKEM parameter set, despite reaching Core-SVP security levels far beyond the NIST Level 5 target. kaiburr-1792 is smaller in combined pk+ct size than HQC-1 (5,792 vs. 6,674 bytes), and less than a third the size of HQC-5 (21,658 bytes)

Table 3: Public-key and ciphertext sizes across KAIBURR, FrodoKEM [19], and HQC [22, 21]. KAIBURR’s classical/quantum Core-SVP security is as computed with the KYBER team’s scripts (Table 2); FrodoKEM and HQC security categories are as claimed by their designers against NIST’s category 1/3/5 targets and are not directly comparable to the Core-SVP numbers.

Scheme	Security	pk (bytes)	ct (bytes)	pk+ct (bytes)
kaiburr-1792	479 / 434 (class./quant.)	2720	3072	5792
kaiburr-4608	1281 / 1162 (class./quant.)	6944	7296	14240
kaiburr-6144	1717 / 1557 (class./quant.)	9248	9600	18848
FrodoKEM-640	Category 1	9616	9720	19336
FrodoKEM-976	Category 3	15632	15744	31376
FrodoKEM-1344	Category 5	21520	21632	43152
HQC-1	Category 1	2241	4433	6674
HQC-3	Category 3	4514	8978	13492
HQC-5	Category 5	7237	14421	21658

4.5 The ceiling of the family

The family $f(t)$ is not unbounded. The lightest noise it can produce is its limit $f(\infty)$, with $\Pr[\pm 1] = \frac{1}{4}$, $\Pr[0] = \frac{1}{2}$, $\Pr[\pm 2] = 0$, and minimal variance $\sigma^2 = 0.5$. At $n = 256$ and $q = 3329$, the failure probability crosses 2^{-128} at $k = 30$, so $k = 29$ is the largest rank that $f(t)$ can support (Table 4). Reaching higher security levels would require another change, such as modifying q , n , or the noise family itself, moving beyond the KYBER structure we seek to preserve.

Table 4: Failure probability under the limiting distribution $f(\infty)$ (minimum variance $\sigma^2 = 0.5$) as k increases, uncompressed. The 2^{-128} bound is first violated at $k = 30$, so $k = 29$ is the ceiling of the $f(t)$ family at $n = 256$, $q = 3329$.

k	Failure probability	pk (bytes)	ct (bytes)
24	$2^{-158.7}$	9248	9600
26	$2^{-146.2}$	10016	10368
28	$2^{-135.5}$	10784	11136
29	$2^{-130.6}$	11168	11520
30	$2^{-126.1}$	11552	11904

5 Implementation

We provide three implementations of KAIBURR, each forked from an existing ML-KEM implementation: a C implementation, a Jasmin implementation with machine-checked constant-time verification, and an assembly implementation for Pavona’s Asymmetric Cryptography Coprocessor (ACC).³ The ACC reuses ML-KEM’s arithmetic core and accelerates Keccak in hardware. This provides us with a complementary hardware-accelerated setting where we can evaluate the cost of scaling the module rank k . As upstream, the C and Jasmin code each have a portable `ref` path and an AVX2-vectorized path; ACC has a single hand-written path.

All three implementations make the same three changes to ML-KEM:

1. the module rank k is scaled to $\{7, 18, 24\}$, giving the parameter sets kaiburr-1792, -4608, and -6144 (named kaiburr4, kaiburr6, and kaiburr8 in the code, after their noise parameter t ; see Table 2);
2. ciphertext compression is removed, so both ciphertext polynomials are serialized at full 12-bit precision; and
3. the centered binomial sampler is replaced by a sampler for the noise family $f(t)$, with $t \in \{4, 6, 8\}$.

Everything else, including the ring, the modulus $q = 3329$, the NTT and its constants, the Fujisaki–Okamoto transform, and the symmetric primitives, is left unchanged.

These modifications nonetheless require more than just mechanical changes to the code. ML-KEM implementations assume $k \leq 4$ in several places, and the AVX2 packers assume a particular coefficient order. Most of the rewritten code follows from scaling k , which we describe first (Section 5.1). We then cover the noise sampler (Section 5.2), the removal of compression (Section 5.3), and what is checked about each implementation (Section 5.4).

³<https://github.com/pavona/pavona>

5.1 Scaling k

Accumulator overflow. Larger values of k can overflow the matrix-vector product. In the NTT domain, each polynomial of the product is a sum of k pointwise products, which ML-KEM accumulates in an `int16_t` and reduces once at the end. In the C reference, the C AVX2, and the Jasmin reference code, each output coefficient is the sum of two separately Montgomery-reduced products, so it is bounded by $2(q-1) = 6656$ rather than by q , and the sum of k terms fits only while $2kq < 2^{15}$, i.e. $k \leq 4$. On the other hand, the Jasmin AVX2 code accumulates both products at 32 bits and applies a single Montgomery reduction, so its terms are bounded by q and it is safe to $k \leq 9$; it also fails at $k = 18$ and $k = 24$.

The C and Jasmin implementations therefore Barrett-reduce the accumulator after every pointwise product, at a cost of one extra reduction per term.⁴ ACC needs no such change since it accumulates with `bn.addvm`, a modular addition against the MOD register, which is set to $2q$ throughout the basemul. Its accumulator stays below $2q$ for every k , and the reduction is free.

Matrix generation. The AVX2 matrix sampler in ML-KEM hardcodes how the k^2 matrix entries are grouped into batches of four for the 4-way Keccak for each $k \leq 4$. We replace this with a generic loop; it samples four entries at a time with 4-way Keccak and handles any remaining entries with a scalar tail. Only $k = 7$ has a remainder, of one entry.

The same loop generates both $\mathbf{A}$ for key generation and its transpose $\mathbf{A}^T$ for encapsulation, by selecting the matching domain separators. The C code computes these separators from the row and column indices, while the Jasmin code reads them from a precomputed table. ACC generates one entry at a time and derives each separator from a running nonce, with separate handling for the transposed and non-transposed orders.

Memory. The C and Jasmin code store the full matrix $\mathbf{A}$ on the stack. Its size grows quadratically with k , from 24.5 KiB at $k = 7$ to 288 KiB at $k = 24$. This is not suitable for ACC, whose stock configuration only provides 32 KiB of data memory. That is less than the working set of a single decapsulation at $k = 24$, which is about 34 KiB for the ciphertext, re-encrypted ciphertext, secret key, and hashing buffers alone, before the matrix is counted.

We address this in two ways. First, the ACC implementation never stores $\mathbf{A}$. It generates one polynomial at a time, multiplies it into a running accumulator, and discards it. Second, we enlarge ACC's visible data memory from 31 KiB to 127 KiB in the hardware description, out of 128 KiB of physical DMEM including a 1 KiB scratch region. The ACC results in Section 7 are therefore for this modified hardware configuration.

5.2 The noise sampler

The noise family $f(t)$ maps t uniformly random bits $a_1, \dots, a_t$, read least-significant first, to the coefficient

$$(2a_t - 1) \left(1 - a_1 + 2 \prod_{i < t} a_i \right).$$

All our implementations replace the centered binomial sampler with this expression, and the PRF output grows to $t \cdot n/8$ bytes per polynomial. The five variants differ in how they compute two parts of the expression: the product term $\prod_{i < t} a_i$ and the sign $(2a_t - 1)$. Table 5 summarizes their choices.

⁴We note that reducing after every term is more conservative than necessary, since a Barrett reduction leaves headroom for several further terms. We did not implement this, as reduction is not a significant part of the cost.

Table 5: How each implementation computes the product term and applies the sign in $f(t)$.

Variant	Product term	Sign	Word size
C ref	bit-by-bit AND chain	integer multiply	scalar
C AVX2	compare against a mask	conditional lane negate	8–32-bit lanes
Jasmin ref	carry propagation	conditional two’s-complement	scalar
Jasmin AVX2	compare against a mask	conditional two’s-complement	8–16-bit lanes
ACC	carry propagation	modular subtract + select	16-bit lanes

The Jasmin reference code and the ACC assembly avoid the literal AND chain with a carry trick. Adding one to the low $t - 1$ bits carries into bit $t - 1$ exactly when all of those bits are set, so a right shift gives us the product. The AVX2 kernels instead isolate the same bits and apply a vector compare, which gives the product directly as an all-ones or all-zero mask. In every vectorized and assembly variant, we then apply the sign separately and without branching.

Extracting t -bit fields from the PRF output is simplest for $t = 8$ and $t = 4$, which align with bytes and nibbles, and hardest for $t = 6$, as explained in Section 4. Both AVX2 kernels use an extra shuffle or gather for this step. The C kernel also reads slightly beyond the end of the PRF output, so the buffer includes 32 bytes of trailing slack. ACC lacks a byte-shuffle instruction, so it instead uses a sequence of `bn.rshi` shifts to move the 6-bit fields across register boundaries.

Key generation needs $2k$ noise polynomials. The AVX2 code samples them in groups of four using the 4-way Keccak.

5.3 Removing compression

KAIBURR serializes both ciphertext polynomials $\mathbf{u}$ and $\mathbf{v}$ at full 12-bit precision, so the ciphertext grows to $(k + 1) \cdot 384$ bytes. The public key keeps its size of $384k + 32$ bytes, since ML-KEM never compresses it.

The reference code and ACC reuse their existing 12-bit packer unchanged. The AVX2 code needs a new one. Its existing packer is used only for the public key, which is stored in the permuted coefficient order of the AVX2 NTT, so it applies that permutation while packing. Ciphertext polynomials leave the inverse NTT in standard order. Both AVX2 forks therefore add a packer and unpacker that work in standard order, without the permutation.

The new packer emits 24 bytes per iteration as two 16-byte stores twelve bytes apart, so its final iteration would write past the end of the buffer. The C code stages the final iteration through a scratch buffer, and the Jasmin code handles it as a separate case.

Removing compression also has a consequence beyond engineering. It makes the decryption-failure probability formally provable, which we develop in Section 6.

5.4 Assurance

Jasmin. The Jasmin implementation is a fork of `formosa-mlkem` [7], whose EasyCrypt proofs establish functional correctness for ML-KEM. These proofs do not carry over to KAIBURR, since the sampler, module rank, and serialization all change. What we check is much narrower.

Every exported function carries two annotations. A `ct` annotation is checked by Jasmin’s constant-time checker, for both standard and speculative constant-time. A `safety` annotation establishes that array accesses stay in bounds and that all memory is initialized before use. Both properties are checked for all six builds.

Testing. The C implementation relies on the deterministic round-trip self-tests inherited from its upstream ML-KEM codebase. The Jasmin implementation runs the same kind of self-test, and additionally cross-checks its output against the C implementation, bit-for-bit, across all six builds. The ACC implementation is validated against a Python model based on `kyber-py`⁵ which subclasses `ML_KEM`, overrides the noise sampler, and disables ciphertext compression. For each parameter set, the model generates known-answer tests (KATs) for key generation, encapsulation, and decapsulation (including implicit rejection), as well as round-trip tests. Unlike the Jasmin code, ACC’s constant-time behavior is not machine-checked; the hand-written kernels avoid secret-dependent branches by construction.

6 Correctness

In the previous sections, we discussed removing compression as a design choice with engineering consequences. We now show why that change matters beyond engineering and show that it turns the decryption-failure probability from a heuristic estimate into a formally verified bound.

Why compression is the obstruction. In the noise expression that Barbosa et al. [8] derive for ML-KEM, $\tilde{n} = \langle e, r \rangle - \langle s, e_1 \rangle - \langle s, c_u \rangle + e_2 + c_v$, the rounding errors c_u and c_v are the only source of difficulty. First, because the correctness adversary holds the secret key, no direct MLWE reduction is available which forces a worst-case elimination of c_v and a split into two sub-bounds. Second, c_u ’s own rounding-error distribution is not symmetric and so fails the “good” property their framework requires. This forces an index-dependent, 256-term sum rather than a single closed-form union bound. In the case of FrodoKEM, since it admits no ciphertext compression, the noise term only involves good distributions, which gives a clean union bound with no heuristic assumption. The cost of compression is concrete and quantitative. For ML-KEM-768, the fully provable bound (which pays for compression) is 2^{-80} , while the heuristic bound that just assumes rounding-of-uniform is 2^{-164} , a gap of roughly 84 bits.

Consequence for Kaiburr We adapt the EasyCrypt framework of [8] and instantiate it for KAIBURR which has no ciphertext compression like FrodoKEM. We reuse their theory for combining distributions over a ring and their proofs that the relevant properties are preserved. We also use their reduction to a single coefficient and their OCaml implementation for computing a sound upper bound on the resulting tail probability. KAIBURR retains KYBER as specified in Sections 2–5 in every respect the analysis depends on. The only property of the noise family $f(t)$ the proof uses is that it is “good” in their sense: non-trivial and symmetric around 0 (see Section 4).

Since the rounding terms vanish, the noise collapses to $\tilde{n} = \langle e, r \rangle - \langle s, e_1 \rangle + e_2$. As a result, every coefficient is identically distributed independent of its index, and the union bound becomes a single factor of 256 rather than a sum of 256 distinct terms. Moreover, no additional MLWE term is needed. We note the resulting expression is in fact simpler than the one obtained for FrodoKEM, since $q = 3329$ is prime, so $\mathbb{Z}_q$ is a field and goodness is preserved through inner products with only $\oplus$ required, whereas FrodoKEM’s power-of-two modulus forces $\ominus$ to be retained.

Computing the bound. The tail probability is computed outside EasyCrypt by OCaml code that mirrors the EasyCrypt expression. We report the results for `kaiburr-1792` ($k = 7$), `kaiburr-4608` ($k = 18$), and `kaiburr-6144` ($k = 24$) in Table 6, alongside the failure

⁵<https://github.com/GiacomoPope/kyber-py>

probability obtained for each parameter set from the KYBER team’s estimator scripts (Table 2).

Table 6: Decryption-failure probability for each KAIBURR parameter set: the formally verified bound obtained in this work, and the estimate from the KYBER team’s scripts (Table 2).

Scheme	k	Verified bound (this work)	KYBER-script estimate
kaiburr-1792	7	$2^{-133.3}$	$2^{-133.7}$
kaiburr-4608	18	$2^{-133.8}$	$2^{-134.1}$
kaiburr-6144	24	$2^{-139.6}$	$2^{-139.9}$

To put the numbers in context: for ML-KEM-768, Barbosa et al. [8] compute a heuristic bound of 2^{-164} , while the best bound they can justify against the definition of correctness is only 2^{-80} , a gap of roughly 84 bits. For KAIBURR there is no such gap to close, since removing compression leaves no rounding term for a heuristic assumption to apply to. Table 6 shows our provable bound closely tracks the KYBER team’s own estimate for every parameter set. This shows that the bound is provable as well as tight in practice.

The price KAIBURR pays by removing compression is ciphertext size. However, this additional size is modest in the context of the high-security parameter sets considered in this work; it also remains within our fixed communication budget.

7 Benchmarks

We benchmark the C and Jasmin implementations on three Intel microarchitectures: Skylake, Coffee Lake, and Raptor Lake, with Hyper-Threading and Turbo Boost disabled. For the hybrid Raptor Lake processor, we pin the benchmark to a performance core. All code is compiled using GCC 12.2.0.

We compare the KAIBURR variants with FrodoKEM and HQC. We use the SHAKE-based variant of FrodoKEM as the primary comparison baseline. Both FrodoKEM-SHAKE and KAIBURR use SHAKE-based symmetric primitives. However, FrodoKEM-AES can take advantage of dedicated AES instructions on our Intel processors, which introduces an implementation-specific hardware advantage unrelated to the lattice-based KEM operations we are looking to compare.⁶ We report only FrodoKEM-640, the smallest standardized FrodoKEM parameter set, since it is already substantially more expensive than KAIBURR. The larger FrodoKEM parameter sets have even higher computational costs and do not provide a more informative performance baseline.

All three KEMs are evaluated using a common benchmark driver adapted from the implementation used in [7]. We first perform 1000 untimed warm-up iterations, followed by 10001 timed iterations. We report the median execution time and interquartile range (Q_1 – Q_3). This procedure is repeated three times, and we report the results from the run whose median is itself the median of the three runs, together with its corresponding quartiles. Each iteration verifies that the decapsulated shared secret matches the encapsulated shared secret.

Randomness comes from Linux `getrandom`. FrodoKEM and KAIBURR call it within `keypair` and `enc`, so that syscall is inside the measured region. HQC instead seeds a SHAKE-256 PRNG once, which we seed from `getrandom` before timing begins.

The benchmark results are reported separately for each processor. Results for the Intel Core i7-6700K, Intel Core i7-8700K, and Intel Core i9-13900K are given in Tables 7, 8, and 9, respectively.

⁶Additionally, we do not report a direct cycle-count comparison with KyFrog [26], the KEM closest to

Table 7: Benchmark results in cycles on an Intel Core i7-6700K. We report only the AVX2 implementations. The reference implementations are, on average, TBD percent slower for KAIBURR C and TBD percent slower for KAIBURR Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen	Enc	Dec
kaiburr-1792	C/asm	TBD	TBD	TBD
kaiburr-1792	Jasmin	TBD	TBD	TBD
kaiburr-4608	C/asm	TBD	TBD	TBD
kaiburr-4608	Jasmin	TBD	TBD	TBD
kaiburr-6144	C/asm	TBD	TBD	TBD
kaiburr-6144	Jasmin	TBD	TBD	TBD
FrodoKEM-640	[19]	TBD	TBD	TBD
HQC-1	[22]	TBD	TBD	TBD
HQC-3	[22]	TBD	TBD	TBD
HQC-5	[22]	TBD	TBD	TBD

KAIBURR is faster than FrodoKEM-640 across every operation, parameter set, implementation, and machine. kaiburr-1792 is 21.5–28.1x faster, kaiburr-4608 is 4.2–5.3x faster, and kaiburr-6144 is 2.5–3.1x faster than FrodoKEM-640.

Comparing the KAIBURR C/asm and Jasmin implementations, the faster implementation depends on the microarchitecture. On the i7-8700K, Jasmin is faster than C/asm for every parameter set and every operation, by 5.4–13.1%. On the i9-13900K, this reverses: C/asm is faster than Jasmin for every parameter set and every operation, by 9.2–14.0%.

Across parameter sets, C/asm KeyGen cycle counts scale by a factor of 5.22 (8700K) / 5.32 (13900K) from kaiburr-1792 ($k = 7$) to kaiburr-4608 ($k = 18$), and by a factor of 1.68 (8700K) / 1.69 (13900K) from kaiburr-4608 to kaiburr-6144 ($k = 24$). For reference, k itself grows by factors of 2.57 and 1.33 over these two steps, and k^2 by factors of 6.61 and 1.78.

kaiburr-1792 is faster than HQC-5 in every operation on both machines: by 64.5–89.8% on the 8700K (KeyGen, Enc, Dec) and 67.8–90.1% on the 13900K. Comparing each KAIBURR parameter set to the HQC parameter set of the same index (kaiburr-1792/HQC-1, kaiburr-4608/HQC-3, kaiburr-6144/HQC-5), HQC’s KeyGen is faster than the corresponding KAIBURR KeyGen by a factor of 1.73, 3.79, and 3.11 on the 8700K, and 1.68, 3.61, and 2.90 on the 13900K.

Interquartile ranges are narrow relative to the medians across all measurements, typically within 1–2% of the median (e.g., 1.3–1.4% for kaiburr-1792 C/asm on the 8700K).

We additionally benchmark the ACC implementation described in Section 5 on Pavona’s cycle-accurate RTL simulator for the accelerator. Because the simulator is deterministic for a fixed random seed, repeated runs of the same test are identical in cycles. Therefore unlike the CPU benchmarks above, we report a single exact cycle count per operation rather than a median and interquartile range. We measure each KEM operation within a single keypair–encapsulate–decapsulate roundtrip and record cumulative execution statistics after each operation. The individual costs are then recovered by subtraction: KeyGen = M_1 , Enc = $M_2 - M_1$, and Dec = $M_3 - M_2$. Results are shown in Table 10.

Table 11 shows the three functions accounting for the largest share of cycles in the same roundtrip benchmark, obtained from the simulator’s per-function execution statistics. Across all three parameter sets, matrix generation (`poly_gen_matrix`) dominates,

KAIBURR in principle, since no public hand-optimized implementation of the scheme exists.

Table 8: Benchmark results in cycles on an Intel Core i7-8700K. We report only the AVX2 implementations. The reference implementations are, on average, 540.8 percent slower for KAIBURR C and 653.8 percent slower for KAIBURR Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen		Enc	
kaiburr-1792	C/asm	156156,(155579–157778)	159719,(159235–161416)	173015,(172106–174015)	173015,(172106–174015)
kaiburr-1792	Jasmin	143857,(143402–144559)	144987,(144656–145831)	150435,(149704–151166)	150435,(149704–151166)
kaiburr-4608	C/asm	814685,(812084–817870)	821346,(818719–824533)	848608,(845755–852461)	848608,(845755–852461)
kaiburr-4608	Jasmin	762767,(761322–764429)	765475,(764033–767156)	781079,(779648–782710)	781079,(779648–782710)
kaiburr-6144	C/asm	1367405,(1363808–1371615)	1376530,(1372953–1380951)	1411130,(1407567–1415693)	1411130,(1407567–1415693)
kaiburr-6144	Jasmin	1293036,(1291067–1295424)	1295674,(1293713–1298070)	1313175,(1311174–1315176)	1313175,(1311174–1315176)
FrodoKEM-640	[19]	3851932,(3851203–3859217)	4067952,(4067235–4075634)	4044333,(4043732–4054934)	4044333,(4043732–4054934)
HQC-1	[22]	90229,(90070–90974)	172063,(171888–172892)	404274,(404018–404510)	404274,(404018–404510)
HQC-3	[22]	215176,(214956–215857)	404687,(404324–405517)	857576,(857048–858104)	857576,(857048–858104)
HQC-5	[22]	439786,(439458–440546)	838353,(837433–839669)	1698485,(1697851–1699119)	1698485,(1697851–1699119)

consuming 73–84% of total cycles.

`poly_gen_matrix` expands the public seed into the uniformly random matrix $\mathbf{A}$ via rejection sampling on SHAKE-128 output. Since it dominates the ACC cycle counts, we break down its costs further. The simulator additionally reports, per function, a cycle count broken down by individual instruction. For kaiburr-6144, this instruction-level breakdown of `poly_gen_matrix` shows that rejection sampling is responsible for the vast majority of its cost. Only 464,022 of its 6,235,212 cycles (7.4%, or 6.2% of the entire benchmark) are spent stalled on `bn.wsrr` waiting for the hardware SHAKE-128 core to produce the next squeeze block. The remaining 92.6% is software cost of the rejection-sampling loop itself. 83.6% of `poly_gen_matrix`’s cycles (69.9% of the whole benchmark) are retired arithmetic instructions (`bn.and`, `bn.cmp`, and `bn.sel`) performing the mask/compare/accumulate operations of rejection sampling, and a further 8.5% (7.1% of the whole benchmark) comes from branch overhead (`bne` and `beq`) in the accept/reject loop. Table 12 summarizes this breakdown.

We repeated this instruction-level breakdown for kaiburr-1792 and kaiburr-4608 and found the same pattern. The hardware-Keccak-wait share stays flat (5.5% and 6.0% of each benchmark’s total cycles, respectively, versus 6.2% for kaiburr-6144), while the rejection-sampling arithmetic share grows with the parameter set (60.6% and 67.6%, versus 69.9% for kaiburr-6144). Table 13 gives the full comparison.

8 Discussion

Our results show that KAIBURR remains computationally practical despite much higher security margins. On all our benchmarks, every KAIBURR parameter set is substantially faster than FrodoKEM-640. This holds even for our largest parameter set, kaiburr-6144, which matches FrodoKEM-640’s communication cost while reaching 1717-bit classical and 1557-bit quantum security, and is still roughly 2.5–3 times faster across all operations. In other words, KAIBURR’s *worst* case is comfortably cheaper than FrodoKEM’s *best* case.

A gap this wide exemplifies the computational advantage of retaining module structure. It also shows that the bandwidth separating the two schemes can be spent in more than one way. Even the largest ML-KEM parameter set, ML-KEM-1024, sits 8,048 public-key bytes and 8,152 ciphertext bytes below the smallest FrodoKEM parameter set, FrodoKEM-640.

Table 9: Benchmark results in cycles on an Intel Core i9-13900K. We report only the AVX2 implementations. The reference implementations are, on average, 498.2 percent slower for KAIBURR C and 557.8 percent slower for KAIBURR Jasmin, measured using the geometric mean. Values are reported as median (Q1–Q3).

KEM	Impl.	KeyGen		Enc	
kaiburr-1792	C/asm	121724,(120872–122872)	125204,(124310–126302)	134816,(134058–136000)	134816,(134058–136000)
kaiburr-1792	Jasmin	141364,(140882–142142)	143058,(142620–143842)	148502,(147944–149060)	148502,(147944–149060)
kaiburr-4608	C/asm	648064,(645678–650674)	654366,(651728–657234)	676840,(674446–679234)	676840,(674446–679234)
kaiburr-4608	Jasmin	747376,(745792–749120)	749472,(747868–751250)	764490,(762874–766106)	764490,(762874–766106)
kaiburr-6144	C/asm	1095102,(1091646–1098922)	1097334,(1093566–1101348)	1127588,(1124030–1131146)	1127588,(1124030–1131146)
kaiburr-6144	Jasmin	1272644,(1270318–1275208)	1272480,(1270246–1274906)	1285796,(1283420–1288172)	1285796,(1283420–1288172)
FrodoKEM-640	[19]	3112706,(3107260–3124744)	3239826,(3233936–3249204)	3195408,(3187982–3202834)	3195408,(3187982–3202834)
HQC-1	[22]	72578,(71902–73220)	139896,(138680–141026)	312692,(310798–314586)	312692,(310798–314586)
HQC-3	[22]	179494,(178058–180668)	332806,(329936–335788)	668334,(663984–672684)	668334,(663984–672684)
HQC-5	[22]	377670,(376224–380158)	705038,(700890–709262)	1357572,(1350500–1364644)	1357572,(1350500–1364644)

Table 10: Benchmark results in cycles for the ACC implementation on Pavona’s cycle-accurate RTL simulator. Values are exact cycle counts.

KEM	KeyGen	Enc	Dec
kaiburr-1792	240816	239348	257474
kaiburr-4608	1453490	1424342	1467109
kaiburr-6144	2505793	2449864	2506070

FrodoKEM spends that bandwidth on *removing* algebraic structure; KAIBURR spends it on *retaining the structure* while reaching the security estimates above.

Framed this way, our comparison also raises a practical question: if FrodoKEM-640’s communication and computational costs are already practical enough for conservative deployments, why not spend the same budget on increasing the security margin of ML-KEM? Our results suggest that this is a plausible alternative. Naively increasing the module rank does reach a correctness limit, but with KAIBURR’s modified noise distribution, the same bandwidth budget supports much larger module ranks, much higher security estimates, and up to three times the speed. What the comparison exposes, then, is a tradeoff between two forms of conservatism: removing algebraic structure, or retaining it and spending the resources on security margin instead.

Against HQC the picture is different, since HQC hedges against a different unknown. On cost alone, however, KAIBURR compares favorably in terms of performance. kaiburr-1792 is smaller than HQC-1 in combined pk+ct size and faster than HQC-5 in every operation on both machines, despite security estimates far beyond the category-5 target. HQC’s advantage is concentrated in key generation, where every HQC parameter set beats the corresponding KAIBURR parameter set by a wide margin. On efficiency grounds, therefore, the margin hedge and the diversity hedge are not in tension, and a deployment that wants both can afford both.

Scaling the module rank also changes where the time goes, and our ACC results show how. Matrix generation accounts for 73–84% of total cycles, and its cost grows quadratically with k , since a $k \times k$ matrix requires k^2 sampled polynomials. Hardware SHAKE-128 acceleration does not remove this bottleneck since the core waits on the accelerator for only 5.5–6.2% of total cycles, with little change across parameter sets.

Table 11: The three functions with the largest share of total cycles in the ACC roundtrip benchmark, as a percentage of total cycles for that parameter set.

KEM	#1	#2	#3
kaiburr-1792	<code>poly_gen_matrix</code> (72.60%)	<code>basemul_acc</code> (8.08%)	<code>f4</code> (6.26%)
kaiburr-4608	<code>poly_gen_matrix</code> (80.78%)	<code>basemul_acc</code> (9.23%)	<code>f6</code> (3.96%)
kaiburr-6144	<code>poly_gen_matrix</code> (83.56%)	<code>basemul_acc</code> (9.57%)	<code>f8</code> (2.05%)

Table 12: Cycle breakdown of `poly_gen_matrix` for kaiburr-6144, obtained from the simulator’s per-instruction execution statistics. Percentages are of the whole kaiburr-6144 roundtrip benchmark (7,461,727 cycles).

Source	Cycles	% of benchmark
Rejection sampling	5,212,503	69.9%
Branch overhead	529,311	7.1%
Keccak wait	464,022	6.2%
Memory latency	29,376	0.4%

The software rejection-sampling loop grows to 69.9% of total cycles for kaiburr-6144. For an accelerated implementation, the optimization target then moves from symmetric throughput to rejection sampling. This is consistent with prior work by Abdulrahman et al., who also identify rejection sampling as a target for further hardware optimization [1]; in our setting, its cost grows with k until it dominates matrix generation.

As discussed in Section 1, our work explicitly explores increasing the security margin as a hedge against improvements in the best known attacks today. By this, we mean that KAIBURR is and will remain vulnerable to structural attacks that exploit the module structure, a class of attacks FrodoKEM is immune against. Such a breakthrough would still motivate an unstructured scheme, so the two approaches remain complementary rather than substitutes. In principle, they could even be combined in a hybrid construction, deriving the final key from both a high-margin module KEM such as KAIBURR and an unstructured LWE KEM such as FrodoKEM. Such a construction would combine the security-margin and structural-diversity hedges, at the cost of increased communication and computation.

The purpose of designing and evaluating these high security variants of ML-KEM is not to argue that such security margins are currently necessary. Considering the relative stability of lattice cryptanalysis and security estimates in recent years, we do not expect improvements of the magnitude that would require such parameter sets in the near future. What we characterize instead is how much margin ML-KEM can accommodate and it costs to obtain it, so that the tradeoff is already understood if it ever becomes necessary.

Two directions follow from this work. Our noise family $f(t)$ is not specific to ML-KEM, so it is an open question whether the same construction could extend the security margin of other module-LWE schemes, such as ML-DSA. On the implementation side, our ACC results leave dedicated hardware support for rejection sampling as the natural next step, since that is what scaling k makes the bottleneck.

Acknowledgements

Large parts of the work were done while the first author was employed at MPI-SP.

Table 13: Cycle breakdown of `poly_gen_matrix` across parameter sets, as a percentage of each variant’s total roundtrip benchmark cycles.

Source	kaiburr-1792	kaiburr-4608	kaiburr-6144
Rejection sampling	60.6%	67.6%	69.9%
Branch overhead	6.2%	6.8%	7.1%
Keccak wait	5.5%	6.0%	6.2%
Memory latency	0.3%	0.4%	0.4%

References

- [1] Abdulrahman, A., Oberhansl, F., Pham, H.N.H., Philipoom, J., Schwabe, P., Stelzer, T., Zankl, A.: Towards ML-KEM & ML-DSA on OpenTitan. In: Blanton, M., Enck, W., Nita-Rotaru, C. (eds.) 2025 IEEE Symposium on Security and Privacy. pp. 1–19. IEEE Computer Society Press (May 2025). doi.org/10.1109/SP61157.2025.002205, 16
- [2] Alagic, G., Bros, M., Ciadoux, P., Cooper, D., Dang, Q., Dang, T., Kelsey, J., Lichtinger, J., Liu, Y.K., Miller, C., Moody, D., Peralta, R., Perlner, R., Robinson, A., Silberg, H., Smith-Tone, D., Waller, N.: Status report on the fourth round of the nist post-quantum cryptography standardization process. Tech. Rep. NIST IR 8545, National Institute of Standards and Technology (Mar 2025). doi.org/10.6028/NIST.IR.8545, <https://csrc.nist.gov/pubs/ir/8545/final> 1
- [3] Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *J. Math. Cryptol.* **9**(3), 169–203 (2015), <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml> 2, 4
- [4] Almeida, J.B., Barbosa, M., Barthe, G., Blot, A., Grégoire, B., Laporte, V., Oliveira, T., Pacheco, H., Schmidt, B., Strub, P.Y.: Jasmin: High-assurance and high-speed cryptography. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017. pp. 1807–1823. ACM Press (Oct / Nov 2017). doi.org/10.1145/3133956.3134078 6
- [5] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber — algorithm specifications and supporting documentation. Submission to the NIST Post-Quantum Cryptography Standardization Project (Nov 2017), <https://pq-crystals.org/kyber/data/kyber-specification.pdf>, round 1 submission, 2017-11-30 4
- [6] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber (version 2.0) — algorithm specifications and supporting documentation. Submission to round 2 of the NIST Post-Quantum Cryptography Standardization Project (Mar 2019), <https://pq-crystals.org/kyber/data/kyber-specification-round2.pdf>, round 2 submission, 2019-03-30 5
- [7] Bacelar Almeida, J., Barbosa, M., Barthe, G., Grégoire, B., Laporte, V., Lécenet, J.C., Oliveira, T., Pacheco, H., Quaresma, M., Schwabe, P., Séré, A., Strub, P.Y.: Formally verifying Kyber: Episode IV: Implementation correctness. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2023**(3), 164–193 (Jun 2023). doi.org/10.46586/tches.v2023.i3.164-193, <https://tches.iacr.org/index.php/TCHES/article/view/10960> 5, 10, 12

- [8] Barbosa, M., Kannwischer, M.J., Lim, T., Schwabe, P., Strub, P.: Formally verified correctness bounds for lattice-based cryptography. In: Huang, C., Chen, J., Shieh, S., Lie, D., Cortier, V. (eds.) Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025. pp. 156–169. ACM (2025). doi.org/10.1145/3719027.3765218, <https://doi.org/10.1145/3719027.3765218> 3, 11, 12
- [9] Barker, E.: Recommendation for key management: Part 1 — general. NIST Special Publication 800-57 Part 1, Revision 5, National Institute of Standards and Technology (May 2020). doi.org/10.6028/NIST.SP.800-57pt1r5 2
- [10] Bernstein, D.J., Chuengsatiansup, C., Lange, T., van Vredendaal, C.: NTRU prime: Reducing attack surface at low cost. In: Adams, C., Camenisch, J. (eds.) Selected Areas in Cryptography - SAC 2017 - 24th International Conference, Ottawa, ON, Canada, August 16-18, 2017, Revised Selected Papers. Lecture Notes in Computer Science, vol. 10719, pp. 235–260. Springer (2017). doi.org/10.1007/978-3-319-72565-9_12 1
- [11] Bos, J.W., Costello, C., Ducas, L., Mironov, I., Naehrig, M., Nikolaenko, V., Raghunathan, A., Stebila, D.: Frodo: Take off the ring! practical, quantum-secure key exchange from LWE. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 1006–1018. ACM (2016). doi.org/10.1145/2976749.2978425 1, 2
- [12] Bos, J.W., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS - kyber: A CCA-secure module-lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018. pp. 353–367. IEEE (2018). doi.org/10.1109/EuroSP.2018.00032 1, 2
- [13] Carrier, K., Meyer-Hilfiger, C., Shen, Y., Tillich, J.: Assessing the impact of a variant of matzov’s dual attack on kyber. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I. Lecture Notes in Computer Science, vol. 16000, pp. 444–476. Springer (2025). doi.org/10.1007/978-3-032-01855-7_15, https://doi.org/10.1007/978-3-032-01855-7_15 2
- [14] Chen, Y., Nguyen, P.Q.: Bkz 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) Advances in Cryptology – ASIACRYPT 2011. pp. 1–20. Springer Berlin Heidelberg, Berlin, Heidelberg (2011) 2, 4
- [15] D’Anvers, J.P., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) AFRICACRYPT 18. LNCS, vol. 10831, pp. 282–305. Springer, Cham (May 2018). doi.org/10.1007/978-3-319-89339-6_16 1
- [16] D’Anvers, J.P., Rossi, M., Virdia, F.: (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part III. LNCS, vol. 12107, pp. 3–33. Springer, Cham (May 2020). doi.org/10.1007/978-3-030-45727-3_1 4
- [17] Ducas, L., Esser, A., Etinski, S., Kirshanova, E.: Asymptotics and improvements of sieving for codes. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part VII. LNCS, vol. 14657, pp. 151–180. Springer, Cham (May 2024). doi.org/10.1007/978-3-031-58754-2_6 1

- [18] Ducas, L., Pulles, L.N.: Does the dual-sieve attack on learning with errors even work? In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. Lecture Notes in Computer Science*, vol. 14083, pp. 37–69. Springer (2023). doi.org/10.1007/978-3-031-38548-3_2, https://doi.org/10.1007/978-3-031-38548-3_2 2
- [19] FrodoKEM Team and Microsoft Research: PQCrypto-LWEKE: FrodoKEM. <https://github.com/microsoft/PQCrypto-LWEKE>, gitHub repository 7, 13, 14, 15
- [20] Guo, Q., Johansson, T.: Faster dual lattice attacks for solving LWE with applications to CRYSTALS. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 13093, pp. 33–62. Springer (2021). doi.org/10.1007/978-3-030-92068-5_2, https://doi.org/10.1007/978-3-030-92068-5_2 2
- [21] HQC Resources Team: Hqc specifications. Documentation PDF (August 2025), <http://pqc-hqc.org>, accessed: 2026-09-16 1, 7
- [22] HQC Team: HQC: Hamming Quasi-Cyclic. <https://gitlab.com/pqc-hqc/hqc>, git-Lab repository 7, 13, 14, 15
- [23] ISO/IEC: Information technology — security techniques — encryption algorithms — part 2: Asymmetric ciphers — amendment 2. Standard ISO/IEC 18033-2:2006/Amd 2:2026, International Organization for Standardization, Geneva, Switzerland (Jun 2026), <https://www.iso.org/standard/86890.html> 1
- [24] Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. *DCC* 75(3), 565–599 (2015). doi.org/10.1007/s10623-014-9938-4 4
- [25] MATZOV: Report on the security of LWE: Improved dual lattice attack. Report, Zenodo (Apr 2022). doi.org/10.5281/zenodo.6493704 2
- [26] Melo, V.D., Buchanan, W.J.: Kyfrog: A high-security lwe-based kem inspired by ml-kem (2025), <https://arxiv.org/abs/2512.06411> 2, 12
- [27] National Institute of Standards and Technology: Post-quantum cryptography: Round 3 submissions. <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> (2020), accessed: 2026-07-27 1
- [28] National Institute of Standards and Technology: Module-lattice-based digital signature standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 204, U.S. Department of Commerce, Washington, D.C. (Aug 2024). doi.org/10.6028/NIST.FIPS.204, <https://doi.org/10.6028/NIST.FIPS.204> 1
- [29] National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 203, U.S. Department of Commerce, Washington, D.C. (Aug 2024). doi.org/10.6028/NIST.FIPS.203, <https://doi.org/10.6028/NIST.FIPS.203> 1, 3, 4
- [30] National Institute of Standards and Technology: Stateless hash-based digital signature standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 205, U.S. Department of Commerce, Washington, D.C. (Aug 2024). doi.org/10.6028/NIST.FIPS.205, <https://doi.org/10.6028/NIST.FIPS.205> 1

- 699 [31] NIST PQC Forum: How is hqc substantially different from rlwe? NIST PQC Forum dis-
700 cussion (Oct 2025), [https://groups.google.com/a/list.nist.gov/g/pqc-forum/](https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/PBC-KPOCSF0)
701 [c/PBC-KPOCSF0](https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/PBC-KPOCSF0), accessed August 29, 2026 1
- 702 [32] PQ Code Package: mlkem-native: A unified, fast and verified ml-kem library. [https:](https://github.com/pq-code-package/mlkem-native)
703 [//github.com/pq-code-package/mlkem-native](https://github.com/pq-code-package/mlkem-native) (2026), gitHub repository, accessed
704 September 17, 2026 5
- 705 [33] The FrodoKEM Team: FrodoKEM: Practical quantum-secure key encapsulation
706 from generic lattices. <https://frodokem.org/#news> (2026), news section. Accessed:
707 2026-07-27 1